

Análisis de consultas SQL en la base dvdrental

Carlos Alexis Vega Torres

Introducción

El presente reporte tiene como objetivo aplicar consultas SQL sobre la base de datos *dvdrental* para responder preguntas específicas de negocio y análisis.

Observación

La base de datos *dvdrental* contiene información de películas, actores, clientes, tiendas y rentas. Se observa que es posible responder preguntas de gestión y análisis mediante consultas SQL.

Planteamiento del problema

Se requiere obtener respuestas a 15 preguntas relacionadas con actores, películas, clientes y ventas, utilizando exclusivamente consultas SQL.

Hipótesis

Si se aplican correctamente las consultas SQL, entonces será posible:

Identificar actores y películas según criterios específicos.

Detectar clientes y patrones de renta.

Calcular métricas de negocio como ventas totales por tienda.

Experimentación

Se ejecutaron 15 consultas SQL sobre la base de datos *dvdrental*. Cada consulta fue diseñada para responder directamente a las preguntas planteadas.

Resultados

1. Se listaron nombres y apellidos de todos los actores.
2. Se obtuvo el nombre completo de cada actor en una sola columna.
3. Se identificaron actores cuyo nombre inicia con "D".

4. Se detectaron actores con nombres repetidos.
5. Se encontró que el costo máximo de renta es 4.99.
6. Se listaron las películas con ese costo máximo.
7. Se contó el número de películas por tipo de audiencia (rating).
8. Se identificaron películas sin rating R o NC-17.
9. Se obtuvo la cantidad de clientes por tienda.
10. Se determinó que la película más rentada fue Bucket Brotherhood (34 veces).
11. Se listaron películas que nunca fueron rentadas.
12. Se identificaron clientes que nunca han rentado.
13. Se encontraron actores con más de 30 películas en su historial.
14. Se calcularon las ventas totales por tienda:
Tienda 1: 30,252.12
Tienda 2: 31,059.92
15. Se listaron clientes que rentaron más de una vez, destacando casos con más de 40 rentas.

Análisis

El análisis de los resultados confirma que:

La base de datos refleja correctamente las relaciones entre actores, películas y clientes.

Las funciones de agregación y filtros permiten responder preguntas de negocio.

Los clientes frecuentes representan un segmento clave para la tienda.

Las ventas se distribuyen de manera equilibrada entre las dos tiendas.

Conclusiones

Se validó que las 15 consultas cumplen con lo solicitado en la actividad.

El archivo SQL contiene la metodología aplicada, mientras que este reporte PDF presenta los resultados y conclusiones.

Se demuestra que SQL es una herramienta eficaz para el análisis de datos en contextos de negocio.

```
/* -----  
* ----- DQL - Data Query Languaje -----  
* by Ivan Alducin  
*/
```

--1. Vamos a seleccionar el nombre y apellido de los actores

```
SELECT first_name, last_name  
FROM actor;
```

--2. Vamos a seleccionar el nombre completo del actor en una sola columna

```
SELECT CONCAT(first_name, ' ', last_name) AS nombre_completo  
FROM actor;
```

--3. Selecciona los actores que su nombre empieza con "D"

```
SELECT first_name, last_name  
FROM actor  
WHERE first_name LIKE 'D%';
```

--4. ¿Tenemos algún actor con el mismo nombre?

```
SELECT first_name, COUNT(*) AS cantidad  
FROM actor  
GROUP BY first_name  
HAVING COUNT(*) > 1;
```

--5. ¿Cuál es el costo máximo de renta de una película?

```
SELECT MAX(rental_rate) AS costo_maximo  
FROM film;
```

--6. ¿Cuáles son las películas que fueron rentadas con ese costo?

```
SELECT title, rental_rate  
FROM film  
WHERE rental_rate = (SELECT MAX(rental_rate) FROM film);
```

--7. ¿Cuántas películas hay por el tipo de audiencia (rating)?

```
SELECT rating, COUNT(*) AS cantidad  
FROM film  
GROUP BY rating;
```

--8. Selecciona las películas que no tienen un rating R o NC-17

```
SELECT title, rating  
FROM film  
WHERE rating NOT IN ('R', 'NC-17');
```

--9. ¿Cuántos clientes hay en cada tienda?

```
SELECT store_id, COUNT(*) AS cantidad_clientes  
FROM customer
```

GROUP BY store_id;

--10. ¿Cuál es la película que mas veces se rento?

```
SELECT TOP 1 f.title, COUNT(r.rental_id) AS veces_rentada
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
GROUP BY f.title
ORDER BY veces_rentada DESC;
```

--11. ¿Qué películas no se han rentado?

```
SELECT f.title
FROM film f
LEFT JOIN inventory i ON f.film_id = i.film_id
LEFT JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.rental_id IS NULL;
```

--12. ¿Qué clientes no han rentado ninguna película?

```
SELECT c.first_name, c.last_name
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id
WHERE r.rental_id IS NULL;
```

--13. ¿Qué actores han actuado en más de 30 películas?

```
SELECT a.first_name, a.last_name, COUNT(fa.film_id) AS cantidad_peliculas
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(fa.film_id) > 30;
```

--14. Muestra las ventas totales por tienda

```
SELECT s.store_id, SUM(CAST(p.amount AS DECIMAL(10,2))) AS ventas_totales
FROM payment p
JOIN staff st ON p.staff_id = st.staff_id
JOIN store s ON st.store_id = s.store_id
GROUP BY s.store_id;
```

--15. Muestra los clientes que rentaron una película más de una vez

```
WITH a AS (
    SELECT c.customer_id, c.first_name, c.last_name, COUNT(r.rental_id) AS
veces_rentadas
    FROM customer c
    JOIN rental r ON c.customer_id = r.customer_id
    GROUP BY c.customer_id, c.first_name, c.last_name
)
SELECT first_name, last_name, veces_rentadas
FROM a
WHERE veces_rentadas > 1;
```